Argus - An AI-powered system that extracts events, reminders, and scheduling information from WhatsApp messages.

> *Named after Argus Panoptes, the all-seeing giant of Greek mythology who never slept, always watching and remembering.*

Current Version: v0.5.0 (Auto-Learning System)

Key Features

- Multi-language support (English, Hindi, Tamil, Telugu, Marathi, Bengali, Gujarati)
- Rule Engine with 70+ static patterns
- Auto-Learning System that improves over time
- Real-time metrics and monitoring
- Push notifications for reminders
- Docker-based deployment with health checks

Tech Stack

Core Application (Argus Service)

Component	Technology	Version	Purpose
Runtime	Node.js	22.x	JavaScript runtime
Language	TypeScript	5.8.x	Type-safe JavaScript
Web Framework	Express.js	5.x	HTTP server & routing
Database	SQLite	better-sqlite3 12.x	Event & message storage
Vector Store	FAISS-like	Custom	Semantic similarity search
Schema Validation	Zod	3.24.x	Runtime type validation
Logging	Winston	3.x	Structured logging

Component	Technology	Version	Purpose
WebSocket	ws	8.x	Real-time communication

AI & Machine Learning

Component	Technology	Version	Purpose
LLM Provider	OpenAI API	SDK 4.96.x	GPT models for extraction
Classifier Model	GPT-4o-mini	Latest	Fast event classification
Extractor Model	GPT-4o	Latest	Detailed data extraction
Embedding Model	text-embedding-3-small	Latest	Vector embeddings
Token Counting	tiktoken	1.x	Accurate token estimation
Token Compression	quicksave	-	Context compression

WhatsApp Integration (Evolution API)

Component	Technology	Version	Purpose
WhatsApp API	Evolution API	v2.1.1	WhatsApp Web bridge
Protocol Library	Baileys	Built-in	WhatsApp Web protocol
Database	PostgreSQL	16-alpine	Evolution API storage

Caching & Message Broker

Component	Technology	Version	Purpose
Cache	Redis	7-alpine	Rate limiting & caching
Pub/Sub	Redis	7-alpine	Inter-service messaging

Push Notifications (Webapp)

Component	Technology	Version	Purpose
Push Protocol	Web Push	3.6.x	VAPID-based notifications
Service Worker	Browser API	-	Background notifications
PWA	Manifest	-	Installable web app

Monitoring Stack

Component	Technology	Version	Purpose
Visualization	Grafana	11.0.0	Dashboards & alerts
Log Aggregation	Loki	3.0.0	Centralized logs
Log Collector	Promtail	3.0.0	Log shipping

DevOps & Infrastructure

Component	Technology	Version	Purpose
Containerization	Docker	24.x	Container runtime
Orchestration	Docker Compose	Latest	Multi-container management
Process Manager	dumb-init	Latest	Signal handling in containers
Testing	Vitest	3.x	Unit & integration tests
Mocking	MSW	2.x	HTTP request mocking
API Testing	Supertest	7.x	HTTP assertions

Dependencies Summary

Category	Count	Key Libraries
Production	14	express, openai, better-sqlite3, zod, winston, web-push, ioredis
Development	16	typescript, vitest, tsx, eslint, msw, supertest
Docker Images	7	node, redis, postgres, grafana, loki, promtail, evolution-api

---

Architecture

System Components

```
+-------------------+     +----------------------+     +------------------+
|   WhatsApp App    |<--->|   WhatsApp Cloud     |<--->|  Evolution API   |
|   (User Phone)    |     |   (Meta Servers)     |     |  (Port 8080)     |
+-------------------+     +----------------------+     +--------+---------+
                                                                 |
                                                                 | Webhook POST
                                                                 v
+-------------------------------------------------------------------------------+
|                          ARGUS SERVICE (Port 3000)                            |
|  +-------------+  +-------------+  +-------------+  +-------------+           |
|  | Heuristic   |->| Classifier  |->| Context     |->| Rule Engine |           |
|  | Gate        |  | (GPT-4o-m)  |  | Builder     |  | + Learning  |           |
|  +-------------+  +-------------+  +-------------+  +------+------+           |
|                                                            |                   |
|                    +---------------------------------------+                   |
|                    |                                       |                   |
|                    v High Confidence                       v Low Confidence    |
|            +-------+-------+                       +-------+-------+           |
|            | Event Router  |                       | LLM Extractor |           |
|            +-------+-------+                       | (GPT-4o)      |           |
|                    |                               +-------+-------+           |
|                    v                                       |                   |
|  +-------------+  +-------------+  +-------------+         |                   |
|  | SQLite DB   |  | Vector      |  | Scheduler   |<--------+                   |
|  | (Events)    |  | Store       |  | (Reminders) |                             |
|  +-------------+  +-------------+  +------+------+                             |
+-------------------------------------------|-----------------------------------+
                                            |
                                            v
+-------------------------------------------------------------------------------+
|                             SUPPORTING SERVICES                               |
|  +---------------+  +---------------+  +---------------+  +---------------+   |
|  | Push Webapp   |  | Redis         |  | Grafana       |  | Loki          |   |
|  | (Port 3002)   |  | (Port 6379)   |  | (Port 3001)   |  | (Port 3100)   |   |
```

```
|  +---------------+  +---------------+  +---------------+  +---------------+       |
+-----------------------------------------------------------------------------------+
                                                |
                                                v
                                +------------------------+
                                |   Browser Notification |
                                +------------------------+
```

Pipeline Flow (v0.5.0)

```
WhatsApp Message
    |
Evolution API Webhook
    |
Store Raw Message (SQLite)
    |
Heuristic Gate (Signal Detection)
    | (if signal found)
Small LLM Classifier (gpt-4o-mini)
    | (if relevant)
Context Builder + Token Compression
    |
Rule Engine (Static + Learned Patterns)
    |
    +--[High Confidence]--> Event Router
    |
    +--[Low Confidence]--> Big LLM Extraction (gpt-4o)
                                |
                            Log to llm_extraction_logs
                                |
                            Event Router
    |
[New Event] --> Insert DB --> Generate Embedding --> Store Vectors
[Update Event] --> Vector Search --> Update DB
[Signal Event] --> Dependency Search --> Activate Pending
    |
Notify Orchestrator --> Push Notification
```

Auto-Learning Loop

```
LLM Extractions --> llm_extraction_logs
                            |
                    Pattern Learner (hourly)
                            |
                    Analyze & Validate
                            |
                    learned_patterns DB
                            |
                    Rule Engine Reload (5 min)
                            |
                    Better Rule Matching --> Fewer LLM Calls --> Cost Savings
```

Docker Services

Service	Image	Port	Profile	Purpose
`rmd-service`	Custom (Node.js)	3000	default	Main Argus application

Service	Image	Port	Profile	Purpose
`webapp`	Custom (Node.js)	3002	default	Push notification webapp
`redis`	redis:7-alpine	6379	default	Caching & rate limiting
`loki`	grafana/loki:3.0.0	3100	monitoring	Log aggregation
`promtail`	grafana/promtail:3.0.0	-	monitoring	Log collection
`grafana`	grafana/grafana:11.0.0	3001	monitoring	Dashboards
`evolution-api`	atendai/evolution-api:v2.1.1	8080	full	WhatsApp bridge
`postgres`	postgres:16-alpine	5432	full	Evolution API database

Docker Profiles

```
# Core services only (Argus + Webapp + Redis)
docker compose up -d

# With monitoring (+ Grafana, Loki, Promtail)
docker compose --profile monitoring up -d

# With WhatsApp integration (+ Evolution API, PostgreSQL)
docker compose --profile full up -d

# Everything
docker compose --profile all up -d
```

Project Structure

```
WHATSAPP-CHAT-RMD/
├── aidata/
│   └── prompt.md                # AI pipeline specification
├── src/
│   ├── index.ts                 # Main entry point
│   ├── server.ts                # Express server (v0.5.0)
│   ├── shared/
│   │   ├── types.ts             # Zod schemas & shared types
│   │   └── utils.ts             # Shared utilities
│   ├── config/
│   │   └── index.ts             # Configuration management
│   ├── orchestrator/
│   │   ├── index.ts             # Orchestrator main
│   │   ├── containerManager.ts  # Docker container management
│   │   ├── notificationService.ts # Web Push notifications
│   │   └── dashboardRouter.ts   # Dashboard API
│   ├── pipeline/
│   │   ├── index.ts             # Pipeline orchestration
│   │   ├── heuristicGate.ts     # Signal detection (regional languages)
│   │   ├── classifier.ts        # Small LLM classifier
│   │   ├── contextBuilder.ts    # Context aggregation
│   │   ├── tokenCompressor.ts   # Token compression
│   │   ├── ruleEngine.ts        # Rule-based extraction + dynamic patterns
│   │   ├── extractor.ts         # Big LLM extraction + logging
│   │   ├── patternLearner.ts    # Auto-learning service (v0.5.0)
│   │   ├── intentDetector.ts    # Intent detection
│   │   └── eventRouter.ts       # Event routing logic
│   ├── database/
```

```
│   │   └── sqlite.ts            # SQLite operations + learning tables
│   ├── vector/
│   │   └── faiss.ts             # Vector store
│   ├── scheduler/
│   │   └── index.ts             # Scheduling service
│   ├── notifications/
│   │   └── index.ts             # Notification service
│   ├── webhook/
│   │   └── evolution.ts         # Evolution API webhook
│   └── utils/
│       ├── logger.ts            # Winston logger
│       ├── metrics.ts           # Pipeline metrics
│       └── pipelineLogger.ts    # Pipeline stage logging
├── tests/
│   ├── setup.ts                 # Test setup
│   ├── unit/
│   │   ├── heuristicGate.test.ts    # 27 tests
│   │   ├── classifier.test.ts        # Tests
│   │   ├── tokenCompressor.test.ts   # 11 tests
│   │   ├── extractor.test.ts         # 10 tests
│   │   ├── ruleEngine.test.ts        # 77 tests
│   │   ├── metrics.test.ts           # 51 tests
│   │   ├── patternLearner.test.ts    # 12 tests (v0.5.0)
│   │   └── types.test.ts             # 13 tests
│   └── integration/
│       ├── pipeline.test.ts          # 8 tests
│       ├── api.test.ts               # 17 tests
│       └── database.test.ts          # 6 tests
├── webapp/
│   ├── server.ts                # Push notification server
│   └── public/
│       ├── index.html           # Web UI
│       ├── sw.js                # Service worker
│       └── manifest.json        # PWA manifest
├── docker/
│   ├── Dockerfile               # Main service container
│   ├── Dockerfile.orchestrator  # Orchestrator container
│   ├── Dockerfile.webapp        # Webapp container
│   ├── docker-compose.yml       # Production stack
│   ├── docker-compose.dev.yml   # Development stack
│   └── grafana/                 # Grafana provisioning
├── scripts/
│   ├── start.sh                 # Start all services
│   ├── stop.sh                  # Stop all services
│   ├── whatsapp-login.sh        # WhatsApp QR login
│   └── test-message.sh          # Send test message
├── docs/
│   ├── ARCHITECTURE.md          # Architecture diagrams (Mermaid)
│   └── diagrams/
│       ├── architecture.mmd     # Complete architecture
│       ├── argus-architecture.pdf   # PDF export
│       ├── argus-architecture.svg   # SVG export
│       └── argus-architecture.png   # PNG export
├── data/
│   ├── db/                      # SQLite database files
│   └── vectors/                 # Vector index files
├── INFO.md                      # This file - Project index
├── RULES.md                     # Project rules and guidelines
├── CHANGELOG.md                 # Version changelog
├── README.md                    # Quick start guide
├── package.json
├── tsconfig.json
├── vitest.config.ts
└── .env.example
```

Database Schema

Core Tables

Table	Purpose	Key Columns
`messages`	All received messages	id, chat_id, content, processed, heuristic_passed
`events`	Extracted events/reminders	id, title, start_time, status, confidence
`subscriptions`	Push notification endpoints	id, endpoint, p256dh_key, auth_key
`pipeline_logs`	Pipeline execution logs	id, message_id, stage, duration_ms

Auto-Learning Tables (v0.5.0)

Table	Purpose	Key Columns
`llm_extraction_logs`	LLM extraction history	id, raw_message, extracted_title, confidence
`learned_patterns`	Auto-generated patterns	id, regex_pattern, hit_count, accuracy
`pattern_learning_runs`	Learning job history	id, patterns_added, status

Event Types

Type	Description	Example
`new_event`	New calendar event	"Meeting tomorrow at 3pm"
`reminder`	Simple reminder	"Remind me to call John"
`update_event`	Modify existing event	"Change meeting to 4pm"
`cancel_event`	Cancel an event	"Cancel tomorrow's meeting"
`signal_event`	Conditional trigger	"When it rains, remind me..."
`irrelevant`	Not event-related	"How are you?"

Environment Variables

Required

Variable	Description
`OPENAI_API_KEY`	OpenAI API key for LLM access

Optional - Application

Variable	Default	Description
`PORT`	`3000`	Argus service port
`NODE_ENV`	`development`	Environment mode

Variable	Default	Description
`DATABASE_PATH`	`./data/db/events.db`	SQLite file path
`LOG_LEVEL`	`debug`	Logging verbosity
`TIMEZONE`	`Asia/Kolkata`	Default timezone

Optional - AI Models

Variable	Default	Description
`OPENAI_MODEL_SMALL`	`gpt-4o-mini`	Fast classifier model
`OPENAI_MODEL_BIG`	`gpt-4o`	Detailed extractor model
`TOKEN_THRESHOLD`	`2000`	Compression trigger threshold

Optional - Auto-Learning

Variable	Default	Description
`PATTERN_LEARNING_INTERVAL`	`3600000`	Learning job interval (1 hour)
`METRICS_LOG_INTERVAL`	`300000`	Metrics logging interval (5 min)

Optional - Push Notifications

Variable	Default	Description
`VAPID_PUBLIC_KEY`	(generated)	Web Push public key
`VAPID_PRIVATE_KEY`	(generated)	Web Push private key
`VAPID_EMAIL`	`mailto:admin@example.com`	Contact email

Optional - Evolution API

Variable	Default	Description
`EVOLUTION_API_URL`	`http://localhost:8080`	Evolution API URL
`EVOLUTION_API_KEY`	-	Evolution API authentication

Optional - Docker

Variable	Default	Description
`RMD_PORT`	`3000`	Host port for Argus service
`WEBAPP_PORT`	`3002`	Host port for webapp
`REDIS_PORT`	`6379`	Host port for Redis
`GRAFANA_PORT`	`3001`	Host port for Grafana
`EVOLUTION_PORT`	`8080`	Host port for Evolution API

API Endpoints

Core Endpoints

Method	Path	Description
GET	`/`	Service info & version
GET	`/health`	Health check
POST	`/webhook/evolution`	Evolution API webhook
POST	`/webhook/test`	Test message endpoint
GET	`/webhook/health`	Webhook health check
GET	`/api/events`	List extracted events
GET	`/api/notifications`	Notification history

Metrics Endpoints

Method	Path	Description
GET	`/api/metrics`	Full metrics JSON
GET	`/api/metrics/summary`	Human-readable summary
POST	`/api/metrics/reset`	Reset all metrics

Learning Endpoints (v0.5.0)

Method	Path	Description
GET	`/api/learning/stats`	Pattern learning statistics
GET	`/api/learning/patterns`	List learned patterns
GET	`/api/learning/logs`	LLM extraction logs
POST	`/api/learning/run`	Trigger manual learning run
DELETE	`/api/learning/patterns/:id`	Deactivate a pattern

Dashboard Endpoints

Method	Path	Description
GET	`/api/dashboard/stats`	Dashboard statistics
GET	`/api/data/stats`	Data collection stats

Quick Start

Development

```
# Install dependencies
npm install

# Copy and configure environment
cp .env.example .env
# Edit .env with your OpenAI API key

# Start all services
./scripts/start.sh

# Or manually:
npm run dev              # Argus service (port 3000)
npm run webapp           # Push webapp (port 3002)

# Run tests
npm test -- --run
```

Docker (Production)

```
# Build and start core services
docker compose -f docker/docker-compose.yml up -d

# With WhatsApp integration (full stack)
docker compose -f docker/docker-compose.yml --profile full up -d

# With monitoring
docker compose -f docker/docker-compose.yml --profile monitoring up -d

# Everything
docker compose -f docker/docker-compose.yml --profile all up -d

# View logs
docker compose -f docker/docker-compose.yml logs -f
```

Testing

```
# Run all tests (237 tests)
npm test -- --run

# Run with coverage
npm run test:coverage

# Run specific test file
npm test -- --run tests/unit/ruleEngine.test.ts

# Watch mode
npm test -- --watch
```

Test Coverage

Module	Tests	Description
Rule Engine	77	Pattern matching, multi-language
Metrics	51	Counter, timing, rates
Heuristic Gate	27	Signal detection

Module	Tests	Description
API Integration	17	HTTP endpoints
Types	13	Schema validation
Pattern Learner	12	Auto-learning system
Token Compressor	11	Compression logic
Extractor	10	LLM extraction
Pipeline Integration	8	End-to-end flow
Database Integration	6	SQLite operations
Total	**237**	All passing

Version History

Version	Date	Highlights
v0.5.0	Current	Auto-Learning System, 237 tests
v0.4.1	-	Metrics system (51 tests)
v0.4.0	-	Rule engine, Regional languages (77 tests)
v0.3.0	-	Startup scripts, Push webapp
v0.2.0	-	Multi-container, Redis, WebSocket
v0.1.0	-	Initial pipeline, SQLite, FAISS

Related Documentation

Document	Description
`docs/ARCHITECTURE.md`	Detailed Mermaid diagrams
`docs/diagrams/argus-architecture.pdf`	Printable architecture diagram
`RULES.md`	Development guidelines & constraints
`CHANGELOG.md`	Detailed version history
`aidata/prompt.md`	AI pipeline specification
`README.md`	Quick start guide